

Pruning Granularity Opportunities in the Uniform Backward Pass

gpurec technical note

May 2026

1 Summary

The current backward pruning test is row-based:

$$a_c = \mathbf{1} \left[\max_s |R_{c,s}| \geq \tau \right],$$

where $R = \text{accumulated_rhs}$ is the current adjoint $\partial L / \partial \Pi$, c is a clade row, s is a species, and τ is `pruning_threshold`. The implementation builds `active_mask` from $\max_s |R_{c,s}|$.

The most important distinction is between the pruning criterion and the amount of work actually skipped:

- The default fused uniform path uses the row mask mainly to skip an entire wave when all rows in that wave are inactive.
- If a wave has at least one active row, the default fused path generally still runs the fused wave kernel on the full wave. It does compute `active_idx` for partial waves, but that compact index is not used by the fused wave kernel.
- Experimental device-pruning paths pass the mask into the Triton kernels, so inactive rows and inactive split parents return early, but those paths also launch fixed schedules for waves that the host-pruned path skipped. That made them slower in the 50-family benchmark.

This leads to a clear hierarchy of opportunities:

1. First, make the existing default cheaper and more honest: avoid unused `nonzero` work in the fused path, and distinguish “threshold inactive” from “work actually skipped” in the statistics.
2. Next, test a hybrid mode: keep host whole-wave skipping, but pass `active_mask` into fused kernels for partially active waves.
3. Then move to device-side active worklists, not fixed-schedule masked kernels. Worklists preserve pruning’s ability to skip work while removing CPU scalar decisions.
4. After row-level pruning is real, add split-level and split-side pruning. The post-optimization bottleneck is split-side work, so this is likely more valuable than further tuning the active-mask kernel itself.

2 Current pruning semantics

In `gpurec/core/backward.py`, each backward wave k reads:

$$R_k = \text{accumulated_rhs}[\text{ws:we}].$$

The active row mask is:

```
clade_max = rhs.abs().max(dim=1).values
if use_pruning:
    active_mask = clade_max >= pruning_threshold
else:
    active_mask = clade_max > 0
```

With `use_pruning=False`, the code still skips exact-zero rows/waves. That is exact zero-work gating, not approximate threshold pruning.

In the normal host-pruned path, the wave loop does:

```
active_mask = compute_active_mask(rhs_k)

if not active_mask.any():
    n_waves_skipped += 1
    n_clades_skipped += W
    continue

n_active = int(active_mask.sum().item())
n_clades_skipped += (W - n_active)

if n_active < W:
    active_idx = active_mask.nonzero(as_tuple=True)[0]
    rhs_active = rhs_k[active_idx]
```

For the non-fused fallback, `active_idx` is used to compact the self-loop solve. For the fast fused uniform path, the call is instead:

```
v_k = wave_backward_uniform_fused(
    Pi_star_wave, Pibar_star_wave, ws, W, S,
    dts_r, rhs_k,
    ...,
    active_mask=active_mask_for_kernels, # None in the default host path
)
```

Since `active_mask_for_kernels` is only set in the experimental `GPUREC_BACKWARD_NO_CPU_PRUNING` or `GPUREC_DEVICE.PRUNING` paths, partial inactive rows are generally not masked inside the default fused kernels.

2.1 Implication

The default fused path has two different notions of pruning:

- **Actual compute pruning:** whole inactive waves are skipped.
- **Diagnostic row pruning:** partial inactive rows are counted in `n_clades_skipped`, but the fused kernels still compute them unless the experimental kernel-pruning flags are enabled.

This is not necessarily a correctness bug, because computing small rows is more accurate than pruning them. But it is a profiling and optimization trap: `n_clades_skipped` currently overstates actual work skipped in the fused default.

3 Why whole-wave pruning still matters

Whole-wave pruning is valuable because it skips the entire wave pipeline:

```
active mask
-> DTS forward recompute, if split wave
-> fused self-loop / wave VJP
-> DTS backward accumulation, if split wave
-> uniform Pibar VJP, if split wave
```

The profiling documents show that removing pruning wholesale is very expensive. In the 50-family wave-2 profile:

Variant	Median	Relative to default
Current default	157.329 ms	1.00×
Disable pruning	241.924 ms	1.54× slower

The no-CPU-pruning diagnostic explains the tradeoff:

Metric	Host pruning	Fixed-schedule device pruning
<code>cudaStreamSynchronize</code> calls	318	246
D2H copies	260	188
Wave self-loop launches	36	49
Split launches	33	46
GPU active time	143.400 ms	149.692 ms
Outside-Nsight median	157.329 ms	161.736 ms

So the fixed-schedule device version removed synchronization, but processed more waves and split waves. The extra scheduled work outweighed the sync savings. The conclusion is not that CPU decisions are good; it is that a device-side design must preserve the work-skipping property of host pruning.

4 Granularity levels

4.1 Wave-level pruning

Wave-level pruning uses:

$$A_k = \bigvee_{c \in \text{wave}(k)} a_c.$$

If $A_k = 0$, the whole wave is skipped.

Strengths.

- Very high payoff per decision: one scalar decision skips multiple kernels.
- No complicated DAG reasoning is needed. At the time a wave is processed, all contributions from later waves have already reached its rows.
- It preserves dense, contiguous wave kernels when a wave is active.

Weaknesses.

- One active row keeps the entire wave alive.

- It becomes less effective as more gene trees are batched together. With many families, the probability that at least one row in a global wave is active approaches one.
- It does not exploit partially inactive waves, which are likely common in large heterogeneous batches.

Opportunity.

For the fused default path, if we only intend to do whole-wave pruning, we do not need `active_mask.sum().item()` or `active_mask.nonzero()` in the hot path. A cheaper whole-wave mode could compute only:

$$A_k = \mathbf{1} \left[\max_{c,s} |R_{c,s}| \geq \tau \right].$$

```
# current default fused path does more than it uses
mask = active_rows(rhs)
if not mask.any():
    continue
n_active = int(mask.sum().item())      # mostly statistics
active_idx = mask.nonzero()[0]         # unused by fused wave kernel
v_k = fused_wave(rhs, active_mask=None)

# cheaper whole-wave-only mode
wave_active = active_wave_scalar(rhs)
if not wave_active:
    continue
v_k = fused_wave(rhs, active_mask=None)
```

This would not add new pruning power, but it should reduce active-mask overhead and remove misleading row-pruning statistics. It is a low-risk cleanup.

4.2 Row/clade-level pruning

Row pruning uses the same `active_mask`, but actually applies it inside the kernels. For row c , pruning means:

- skip the wave self-loop VJP for row c ;
- do not accumulate parameter gradients from row c ;
- skip DTS split programs whose parent is c ;
- skip uniform Pibar VJP split sides whose split parent is c .

This is already implemented in the experimental masked kernels. For example, the wave kernel checks:

```
if USE_ACTIVE_MASK:
    row_active = tl.load(active_mask_ptr + w)
    if row_active == 0:
        store_zero_outputs()
    return
```

The DTS and Pibar kernels similarly check `active_mask[reduce_idx[i]]` for each split parent.

4.2.1 Most promising near-term experiment: hybrid host-wave plus row mask

The previous failed experiment was fixed-schedule device pruning: it removed host whole-wave decisions and launched masked kernels for all waves. A more promising experiment is different:

1. Keep the current host whole-wave skip.
2. If the wave is active, pass `active_mask` into the fused wave, DTS, and Pibar kernels.
3. Avoid `active_idx = nonzero(mask)` unless the generic fallback needs compacted rows.

```
# proposed hybrid
mask = active_rows(rhs)
if not mask.any():
    continue

if use_fused:
    v_k = fused_wave(rhs, active_mask=mask)
    dts = dts_fused(..., active_mask=mask)
    dts_backward(..., active_mask=mask)
    pibar_vjp(..., active_mask=mask)
else:
    active_idx = mask.nonzero()[0]
    run_compacted_fallback(active_idx)
```

This preserves the benefit that made host pruning faster than fixed scheduling: all-inactive waves are still skipped. It also gives real row-level pruning inside active waves.

The cost is an extra mask load and branch in the fused kernels. That cost can be controlled by only passing the mask when the inactive fraction is large enough, for example:

$$\frac{W - n_{\text{active}}}{W} \geq \rho.$$

The threshold ρ should be benchmarked. Reasonable first values are 0, 0.1, 0.25, and 0.5.

4.3 Chunk or subwave pruning

Chunk pruning is between whole-wave and row-level pruning. Split a large wave into contiguous row chunks:

$$\text{wave}(k) = B_{k,0} \cup B_{k,1} \cup \dots$$

and compute:

$$A_{k,b} = \bigvee_{c \in B_{k,b}} a_c.$$

This can be useful because the current global wave may be active due to a small number of rows. A chunk-level scheduler can skip inactive chunks while still running dense row-contiguous kernels for active chunks.

Tradeoff.

- Smaller chunks improve pruning granularity.
- Smaller chunks increase launch count, reduce occupancy opportunities, and may fragment split metadata.

The earlier `max_wave_size` sweep is relevant but not decisive: `max_wave_size=16384` was slower than 32768 in the old 50-family run, but that was a static memory cap, not an active chunk scheduler. For 1000 families, chunk-level pruning may be more valuable because whole global waves are less likely to be fully inactive.

5 DAG subtree pruning

It is tempting to say that if a clade is inactive, we can prune its whole subtree. In a tree that would often be correct. In the clade DAG it is not that simple.

A child clade can have multiple incoming split paths. If one parent is inactive, another active parent may still send adjoint to the same child. So we cannot delete a structural descendant subtree just because one ancestor row is inactive.

The safe dynamic rule is:

Process waves in reverse topological order. When a row’s wave is reached, all already-processed parents have accumulated their contributions into that row’s RHS. If the final row RHS is below threshold, that row can be pruned.

This is exactly what row-level RHS pruning approximates. A more aggressive “subtree” implementation should therefore be phrased as an active-dependency worklist, not as a static deletion of descendants:

```
active_rows_k = rows in wave k with absmax(rhs[row]) >= threshold
active_splits_k = splits whose parent row is in active_rows_k

for each active split:
    accumulate into child RHS

# child rows become active later only if their accumulated RHS is large enough
```

In other words, the DAG pruning object is not a subtree. It is the dynamically reachable set of rows under the current adjoint threshold.

6 Split-level pruning

Row-level pruning skips all splits of inactive parents. It does not skip individual negligible splits of an active parent. That matters because the post-optimization profile is again dominated by split-side work:

Component after proposal 6	Approximate Nsys bucket		Notes
<code>_dts_cross_backward_accum_kernel</code>	32.606 ms	split-major	direct adjoints
<code>_uniform_cross_pibar_vjp_tree_from_ud_kernel</code>	25.618 ms		two split sides
<code>_dts_fused_kernel</code>	12.207 ms	DTS forward	recompute

The largest split waves are root-like:

Wave shape	Rows	Splits	Fanout
50-family wave example	247	42155	170.7 splits/parent
50-family wave example	39	24229	621.3 splits/parent

If a parent row is active, row-level pruning keeps all its splits. For these root-like waves, split-level pruning could be much more selective.

6.1 Split predicates

A split i with parent $p(i)$ should at least satisfy:

$$b_i = a_{p(i)}.$$

That is parent-row pruning.

A stronger approximate split predicate would test the actual split responsibility or VJP contribution:

$$b_i = a_{p(i)} \wedge \left[\max_s |v_{p(i),s}| \cdot \omega_{i,s} \geq \tau_{\text{split}} \right],$$

where $\omega_{i,s}$ is the split's normalized contribution to the parent DTS logsumexp for species s .

The challenge is that $\omega_{i,s}$ is not free. It depends on the same DTS terms that the split kernels compute. Therefore split pruning should be introduced in stages:

1. First use parent-active split pruning, because that only needs `active_mask[reduce_idx]`.
2. Then collect statistics on per-split contribution magnitudes inside the existing DTS kernel, without changing results.
3. Only if many active-parent splits have tiny contribution, add a compacted active-split work-list.

6.2 Split-side pruning

The staged `Pibar` path computes the tree input:

$$u_d = \text{grad_Pibar} \cdot \text{inv_denom}, \quad A = \sum_s u_d[s].$$

This creates a natural split-side pruning criterion:

$$d_{i,\text{side}} = \mathbf{1} \left[\max_s |u_{d,i,\text{side},s}| \geq \tau_{\text{side}} \right].$$

If $d_{i,\text{side}} = 0$, the corresponding `Pibar` tree VJP side can be skipped. This is attractive because the current staged tree VJP bucket is about 25.6 ms in the post-proposal-6 profile.

The staged DTS kernel already touches the values needed to compute this predicate. The missing piece is scheduling: the tree kernel should run only on active split sides rather than launching a fixed $2n_{\text{ws}}$ grid.

7 Species-lane or tile pruning

Species-lane pruning would skip individual species entries or species tiles inside an active row. This is the finest granularity:

$$a_{c,\text{tile}} = \mathbf{1} \left[\max_{s \in \text{tile}} |R_{c,s}| \geq \tau_{\text{tile}} \right].$$

This is currently the least attractive pruning level.

- Uniform `Pibar` uses row reductions and species-tree corrections. Dropping lanes changes row sums and ancestor/subtree corrections.
- The wave kernel's expensive work is dense in S , and many formulas use $\sum_s$ or $\max$ over the full row.

- A row can have many species lanes just below a max threshold. Their aggregate contribution can still matter for parameter gradients.
- The active-mask kernel already streams all S lanes once; additional tile metadata may not pay for itself unless rows are extremely sparse in species space.

Species pruning should therefore be considered only after row and split worklists are measured. A safer intermediate is to improve the row predicate, for example by recording both $\|R_c\|_\infty$ and $\|R_c\|_1$, rather than dropping species lanes.

8 Family-level and batched-tree granularity

The current batching strategy builds large global waves across many gene trees. That improves occupancy, but it weakens whole-wave pruning. With many families, a global wave is active if any family contributes an active row:

$$A_k^{global} = \bigvee_g \bigvee_{c \in \text{wave}(k,g)} a_c.$$

As the number of families grows, A_k^{global} tends to be true even if most families are inactive at that depth. This is likely important for 1000-tree batches.

The pruning-aware alternative is to schedule at a family-wave or chunk level:

```
for global wave k:
  build active row counts by family or row chunk
  select active family-wave chunks
  launch kernels only for selected chunks
```

This creates a three-way tradeoff:

- Larger batches improve occupancy and reduce launch overhead.
- Smaller family/chunk units improve pruning selectivity.
- Buffer lifetime and memory pressure constrain how many chunks can be resident at once.

For 1000 trees, this may matter more than further optimizing the row mask kernel. The current active-mask kernel is already a simple bandwidth-streaming kernel; the larger question is whether its output can drive a better scheduler.

9 Recommended optimization sequence

9.1 P0: fix measurement and remove unused default work

Before changing pruning behavior, separate statistics into:

- rows below threshold;
- rows whose self-loop work was actually skipped;
- split programs skipped;
- Pibar split sides skipped;
- whole waves skipped.

Then remove `active_idx = active_mask.nonzero()` from the fused default unless a compact fallback path will actually use it. If production does not need `n_clades_skipped` every iteration, also avoid `active_mask.sum().item()` in the hot path or compute it asynchronously for diagnostics.

9.2 P1: hybrid host-wave skip plus fused row masks

This is the highest-value low-complexity experiment:

```
mask = active_rows(rhs)
if not mask.any():
    skip whole wave
else:
    pass mask into fused wave, DTS forward, DTS backward, and Pibar VJP kernels
```

It should be tested with several policies:

Policy	Meaning
always mask active waves	Maximize row pruning, measure branch overhead
mask if $n_{\text{active}} < W$	Avoid all-active mask overhead
mask if inactive fraction $\geq 10\%$	Avoid overhead on nearly full waves
mask only split waves	Target DTS and Pibar buckets first

This experiment is different from the failed device-pruning prototype because it keeps whole-wave skipping.

9.3 P2: device active row and split worklists

If P1 shows meaningful inactive-row work inside active waves, the next step is a worklist scheduler:

```
row_mask = active_rows(rhs)
active_rows = select(rows, row_mask)
active_splits = select(splits, row_mask[reduce_idx])
active_sides = select(split_sides, row_mask[reduce_idx] & side_active)

wave_kernel_active(active_rows)
dts_kernel_active(active_splits)
pibar_tree_active(active_sides)
```

This can be built with CUB-style prefix sums/select operations or a C++ CUDA extension. The key is to avoid the fixed-schedule failure mode: inactive waves and inactive split groups must not merely early-return after launch; they should be absent from the launched work.

9.4 P3: split-side pruning after instrumentation

Instrument the staged DTS path to record:

- max and sum of $|u_d|$ per split side;
- max VJP contribution per split;
- active split count under candidate thresholds;
- how these counts correlate with final gradient error.

If many split sides are negligible, compact Pibar tree VJP to active sides. This directly targets the roughly 25.6 ms staged tree bucket in the post-proposal-6 profile.

9.5 P4: family/chunk-aware scheduling for large batches

For 1000 trees, record active rows by family and wave. The relevant metric is not only the number of active global waves, but:

$$\frac{\text{active rows in wave}}{\text{rows scheduled in wave}}.$$

If global waves are almost always active but sparse by family or chunk, use family-wave chunks or row chunks as the scheduling unit. This may recover pruning selectivity without sacrificing all batching benefits.

10 Expected gains and risks

Opportunity		Expected gain	Risk	Why
Remove unused <code>nonzero/stats</code> in fused default		Small, likely 1–3 ms if partial waves are common	Low	Does not change numerical pruning semantics if statistics are deferred or corrected.
Host-wave skip plus active mask in fused kernels		Medium, data-dependent	Medium	Preserves whole-wave skip while enabling real row/split-parent pruning; changes approximate pruning results for rows below threshold.
Device worklists	active	Medium to high, especially for 1000 trees	High	Removes CPU syncs and no-op work, but requires robust compaction, buffer management, and irregular-kernel interfaces.
Split-side pruning	Pibar	Potentially high if many sides are tiny	Medium/high	Targets a 25 ms-scale bucket, but needs new contribution thresholds and active-side scheduling.
Family/chunk scheduling		Potentially high for large batches	High	Whole global waves become less selective as family count grows; chunking restores selectivity but complicates memory/lifetime scheduling.
Species-tile pruning		Low/uncertain	High	Uniform Pibar and parameter reductions are dense over species; correctness bounds are harder.

11 Bottom line

The current default mainly prunes whole waves. That is still valuable, but it leaves three forms of work on the table:

1. partially inactive rows inside active waves;
2. inactive or negligible splits of active split waves;
3. inactive family/chunk regions inside large cross-family waves.

The failed device-pruning experiment should not discourage pruning work. It tested the wrong tradeoff: it removed host decisions but gave up whole-wave skipping. The next pruning experiments should preserve whole-wave skipping while adding finer-grained masks, then graduate to device worklists if the active fractions justify the complexity.